A PROJECT REPORT

ON

SHOPSMART

E-COMMERCE WEBSITE

FOR

DATA SCIENCE WITH GENERATIVE AI

BY

SHAIKH REHAN JAMEEL AHMED

DECLARATION

I, **"Shaikh Rehan Jameel Ahmed"** hereby declare that this project report titled **"ShopSmart – E-Commerce Website"** has been prepared for academic purposes. The technical descriptions are based on the supplied source code and SQL file. The interface images are illustrative mockups and are not screenshots captured from a running deployment. I will perform local execution and testing before claiming that the application has passed its tests.

ACKNOWLEDGEMENT

I express my sincere gratitude to my project guide, department and institution for their encouragement and guidance during the preparation of this project. Their suggestions helped organize the project work and its documentation.

I also acknowledge the developers and communities behind PHP, MySQL/MariaDB, HTML, CSS, JavaScript, XAMPP and Visual Studio Code. These technologies provide the foundation for the ShopSmart educational application

ABSTRACT

ShopSmart is a student-oriented e-commerce website demonstration. It models the core activities of an online store: product discovery, customer registration and login, shopping-cart management, checkout through Cash on Delivery, order history and administrator management. PHP handles server-side requests and application logic. PDO provides database access, while MySQL/MariaDB stores user, category, product and order information. The supplied SQL script creates five relational tables and inserts four categories with twelve sample products. Basic safeguards include password hashing, prepared statements, output escaping, CSRF-token validation, session ID regeneration, role checks and transactional stock validation. The application is intended for local academic demonstration, not as a production payment platform.

TABLE OF CONTENTS:

Page	Section
1	Cover Page
2	Certificate and Declaration
3	Acknowledgement and Abstract
4	Table of Contents
5	Introduction and Project Background
6	Problem Statement, Aim and Objectives
7	Scope, Users and Requirements
8	Technology Stack and Requirements
9	System Architecture and Data Flow
10	Project Folder Structure
11	Functional Modules
12	User Workflows and Use Cases
13	Database Overview and Data Dictionary
14	Database Tables and Constraints
15	Database Relationship Diagram
16	Illustrative Screenshot: Home Page
17	Illustrative Screenshot: Product Catalogue
18	Illustrative Screenshot: Customer Login
19	Illustrative Screenshot: Shopping Cart
20	Illustrative Screenshot: COD Checkout
21	Illustrative Screenshot: Admin Dashboard
22	Illustrative Screenshot: Database Diagram
23	Security, Validation and Testing Plan
24	Installation, Execution and Troubleshooting
25	Limitations, Future Scope, Conclusion and References

1. INTRODUCTION AND PROJECT BACKGROUND:-

Electronic commerce allows customers explore products and place orders through a website. A web-based store can make product information easier to access and can centralize customer and order records. ShopSmart demonstrates these ideas in a compact application suitable for learning.

1.1 Project Background

The supplied project contains customer-facing PHP pages, reusable include files, administrative pages, static assets and a database creation script. It is designed to run locally using Apache, PHP and MySQL through a package such as XAMPP.

1.2 Purpose of the project

- Demonstrate the relationship between browser pages, server-side code and a relational database.
- Provide a practical example of account handling and role based-access.
- Show how a cart can be converted into a persistent order.
- Demonstrate stock checking and transactional database updates.
- Provide an admin interface for basic store operations.

1.3 Intended Audience

The primary audience is students and instructors studying web development, PHP, database connectivity and introductory e-commerce workflows. The sample data and COD-only checkout keep the demonstration straightforward.

---

2. PROBLEM STATEMENT, AIM AND OBJECTIVES:-

2.1 Problem Statement:

When catalogue information, customer details and orders are maintained separately, it can be difficult to find current product information, verify stock and follow an order from placement to completion. A small integrated application can demonstrate how these are connected.

2.2 Aim

The aim is to develop and document a database-backed e-commerce website that demonstrates product browsing, customer account management, shopping-cart operations, Cash on Delivery order placement and administrator controls.

2.3 Objectives

- Create a home page with featured products.
- Display an active product catalogue with search and category filtering.
- Allow customers to register, sign in and sign out.
- Maintain a session-based cart and validate quantities against stock.
- Collect delivery information and create COD orders.
- Display customer's order history.
- Provide administrator pages for products, categories and orders.
- Store data using relational tables, primary key and foreign key.
- Apply basic secure coding practices and prepare a manual test plan.

2.4 Expected Outcome

A locally executable learning project in which a customer can complete the sample shopping workflow and an authorized administrator can maintain store records.

---

3. SCOPE, USERS AND FEASIBILITY:-

3.1 Project Scope

The scope includes catalogue browsing, product details, customer registration and login, cart updates, COD checkout, order history and admin management. It includes sample categories and product records. It does not include online card payment or real shipping-providing integration.

3.2 User Roles

Role	Main activities
Visitors	View home page, browse and search products, register or sign in.
Customer	Manage own session cart, submit COD order and view own order history.

Administrator	View dashboard, manage products and categories and update order statuses.

3.3 Feasibility

- Technical feasibility: PHP, MySQL and XAMPP are widely used for local web development.
- Economic feasibility: The project uses common free development tools and does not require a paid payment services.
- Operational feasibility: The browser interface and familiar shopping workflow are suitable for a student demonstration.
- Schedule feasibility: The project is divided into small that can be developed and individually.

3.4 Assumptions

The user has access to a computer, browser and local PHP/MySQL environment. The supplied SQL script is imported before opening the website. Local database credentials may need adjustment.

---

4. TECHNOLOGY STACK AND REQUIREMENTS:-

Component	Technology	Use in project
Front end	HTML	Page structure and forms.
Presentation	CSS	Layout, colors, responsive styling.
Client-side	Vanilla JavaScript	Small browser interactions.
Server-side	PHP	Session, validation, business rules.
Database access	PDO	Prepared SQL statements and database errors.
Database	MySQL / MariaDB	Persistent relational records.
Local server	Apache via XAMPP	Serve PHP pages to the browser.

Editor	Visual Studio Code	Edit and inspect source code.
Browser	Chrome/Edge	Run and test the web application.

4.1 Hardware Requirements

- Desktop or laptop computer capable of running a local web server.
- Sufficient RAM and disk space for XAMPP the project folder and database.
- Keyboard, mouse and a display suitable for browser testing.

4.2 Software Requirements

- Windows 10/11 or another supported operating system.
- XAMPP with Apache, PHP and MySQL/MariaDB.
- Visual Studio Code or an equivalent editor.
- A modern web browser.

4.3 Version Note

PHP 8.1 or later is recommended for the supplied syntax. Confirm the PHP and MySQL/MariaDB versions installed on the target computer before testing.

---

5. SYSTEM ARCHITECTURE AND DATA FLOW:-

ShopSmart follows a simple browser-server-database architecture. The browser requests a PHP page from Apache. PHP loads the database connection and shared helper functions, validates the request, performs the required SQL operations and return HTML to the browser.

Layer	Responsibilities
Representation layer	HTML forms, navigation, product cards, tables and CSS layout.
Application layer	PHP page controllers, authentication, cart rules, checkout validation and admin actions.
Data layer	PDO connection, prepared SQL statements, transaction handling and MySQL tables.
Session layer	Stores the signed-in user, flash messages, CSRF token and cart quantities.

5.1 Order Data Flow

1. Customer signs in and adds available products to the cart.

2. Customer opens checkout and submits delivery details.

3. Server validates from data and verifies the CSRF token.

4. A database transaction begins and each requested product row is locked for stock validation.

5. The order and order-item records are inserted and inventory is reduced.

6. The transaction commits; the session cart is cleared and the customer is redirected to order history.

7. If a failure occurs, the transaction is rolled back and an error is shown.

5.2 Architecture Benefits

Separating reusable database and helper code from individual pages reduces duplication and makes the project easier to maintain. A relational database preserves links between customers, products and orders.

---

6. PROJECT FOLDER STRUCTURE:-

The supplied archive is organized into PHP pages, shared includes, configuration, administration pages, front-end assets and database scripts.

```
E-Commerce-Website/
├── index.php
├── products.php
├── product-details.php
├── cart.php
├── checkout.php
├── orders.php
├── login.php / register.php / logout.php
├── config/
│   └── database.php
├── includes/
│   ├── header.php
│   ├── footer.php
│   └── functions.php
```

```
├── admin/
│   ├── _guard.php
│   ├── dashboard.php
│   ├── products.php
│   ├── categories.php
│   └── orders.php
├── assets/
│   ├── css/style.css
│   ├── js/script.js
│   └── images/
└── database/
    └── ecommerce.sql
```

6.1 Important Shared Files

- config/database.php creates the PDO connection and defines database credentials.
- includes/functions.php contains escaping, redirects, CSRF, login and cart helper functions.
- includes/header.php and footer.php provide a common page layout.
- admin/_guard.php protects administrator pages.
- database/ecommerce.sql creates the database, tables and sample catalogue.

---

7. FUNCTIONAL MODULES:-

Module	Files	Description
Home and catalogue	index.php, products.php	Featured products, search and category filtering.
Products details	product-details.php	Individual product information and add-to-cart action.
Customer accounts	register.php, login.php, logout.php	Create account, verify password, start/end session.
Cart	cart.php, functions.php	Add items, change quantity, remove items and calculate totals.
Checkout	checkout.php	Validate delivery details, confirm stock and create COD order.
Order history	orders.php	Show the signed-in customer's orders.

Admin dashboard	admin/dashboard.php	Display product, order, customer and order-value summaries.
Admin catalogue	admin/products.php admin/categories.php	Maintain product and category records.
Admin orders	admin/orders.php	Review orders and update order status.

7.1 Reusable Functions

The helper file includes output escaping through e(), safe redirect handling, CSRF token generation and verification, current-user lookup, login and administrator guards, flash messages, rupee formatting, cart item counts and cart product retrieval.

7.2 Module Integration

The modules share a common PDO connection and session. A successful customer login makes protected customer actions available; a role check separates administrative actions from normal customer actions.

8. USER WORKFLOWS AND USE CASES:-

8.1 Visitor Workflow

1. Open the website home page.
2. Browse the product catalogue or search by keyword.
3. Open a product detail page.
4. Register or sign in to use account-dependent features.

8.2 Customer Workflow

1. Sign in with a registered account.
2. Add one or more products to the cart.
3. Review items and adjust quantities or remove an item.
4. Continue to checkout.
5. Enter full name, phone number and delivery address.
6. Select the displayed Cash on Delivery method and place the order.
7. View the order in the order-history page.

8.3 Administrator Workflow

1. Register a normal account and promote it to admin in the local database.
2. Sign out and sign in again to refresh the role in the session.
3. Open the admin dashboard.
4. Manage products and categories.
5. Review incoming orders and update their status.

8.4 Main Use Cases

Actor	Use case	Expected response
Visitor	Search products	Matching active products appear.
Customer	Place COD order	Order is stored and stock is updated.
Customer	View order history	Only the customer's own orders are shown.
Admin	Update order status	Order status is saved in the database.

---

9. DATABASE OVERVIEW AND DATA DICTIONARY:-

The supplied SQL files creates the ecommerce_portal database using UTF-8 compatible character settings. It defines five InnoDB tables and uses primary and foreign keys to maintain relationships.

Table	Purpose	Key fields
users	Customer and admin accounts.	id, names, email, password_hash, role, created_at.
categories	Catalogue grouping.	Id, name, created_at.
products	Product catalogue and stock.	Id, category_id, name, description, price, stock, image, active.
orders	Order header and delivery information.	Id, user_id, full_name, phone, address, city, postal_code, total, payment_method, status.

order_items	Individual product lines in an order.	Id, order_id, product_id, product_name, unit_price, quantity, line_total.

9.1 Sample Data

The SQL script inserts four categories: Electronics, Home & Living, Accessories and Fitness. It inserts twelve sample products with example prices and stock quantities. Product images are referenced from the assets/images folder.

9.2 Data Integrity

- The users email field is unique.
- Category names are unique.
- Products can reference a category; deleting the category set the product category reference to NULL.
- Orders reference a user and restrict deletion of a user with existing orders,
- Order items belong to an order and are deleted with that order.
- Order-item product references may become NULL while the product name and price snapshot remain available.

---

10. DATABASE TABLES AND CONSTRAINTS:-

10.1 Users and Categories

The users table stores account identity, email password hash and role. The role is limited to customer or admin. The categories tables stores unique category names.

10.2 Products

Each product contains a name, description, decimal price, unsigned stock quantity, image path and active flag. The category_id field may be NULL. Indexes support product-name lookup and active-product filtering.

10.3 Orders

An order belongs to a registered user and stores delivery name, phone, address, city and postal code. The payment_method fields is restricted to COD. The status field supports placed, processing, shipped, delivered and cancelled.

10.4 Order Items

Each order item stores a product snapshot: product name, unit price, quantity and line total. Saving the name and price with the order line helps preserve what was ordered even if the catalogue changes later.

10.5 Why Decimal Is Used for Money

The schema uses DECIMAL(10,2) for product prices, order totals and line totals. Decimal storage is appropriate for currency values because it avoids the binary rounding behaviour associated with floating-point storage.

---

11. DATABASE RELATIONSHIP DIAGRAM:-

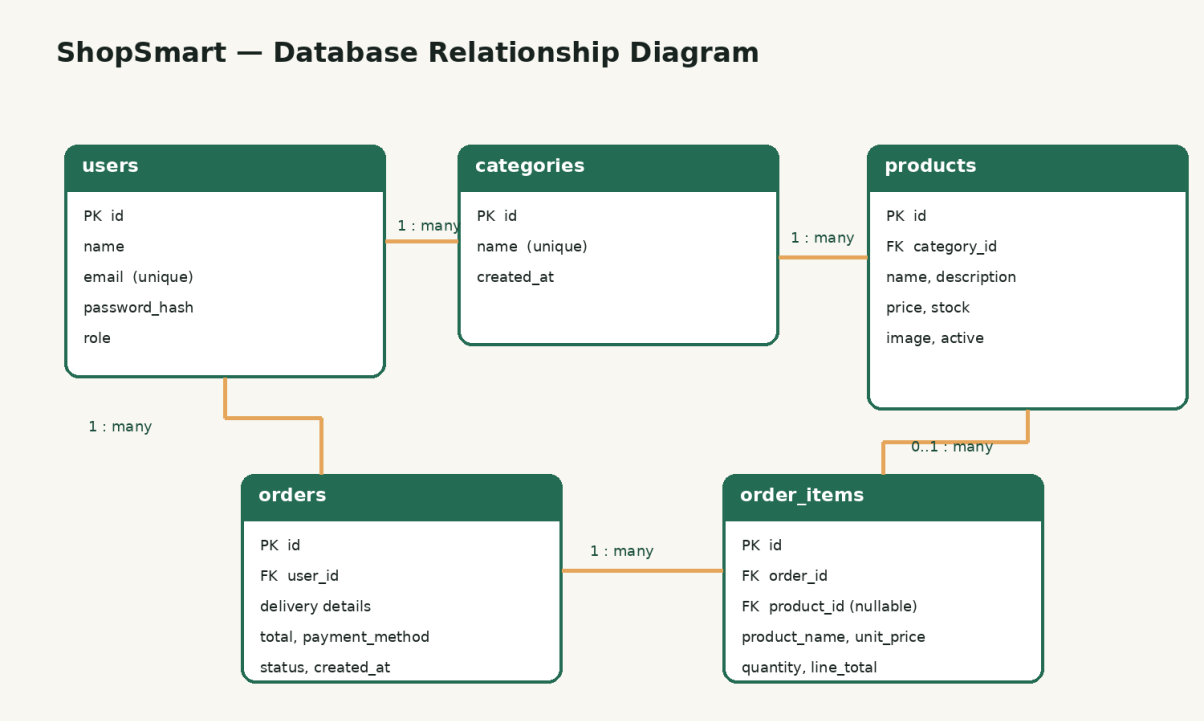


Figure 1. Illustrative entity relationship diagram based on the supplied SQL schema.

11.1 Relationship Explanation

- One category can be associated with many products.
- One user can place multiple orders.
- One order can contain multiple order-item rows.
- An order item may retain is product-name and price snapshot of its product reference is later removed.

The diagram is a simplified logical view of the SQL structure. The actual constraints and deletion behaviour are defined in database/ecommerce.sql.

---

12. ILLUSTRATIVE SCREENSHOT— HOME PAGE:-

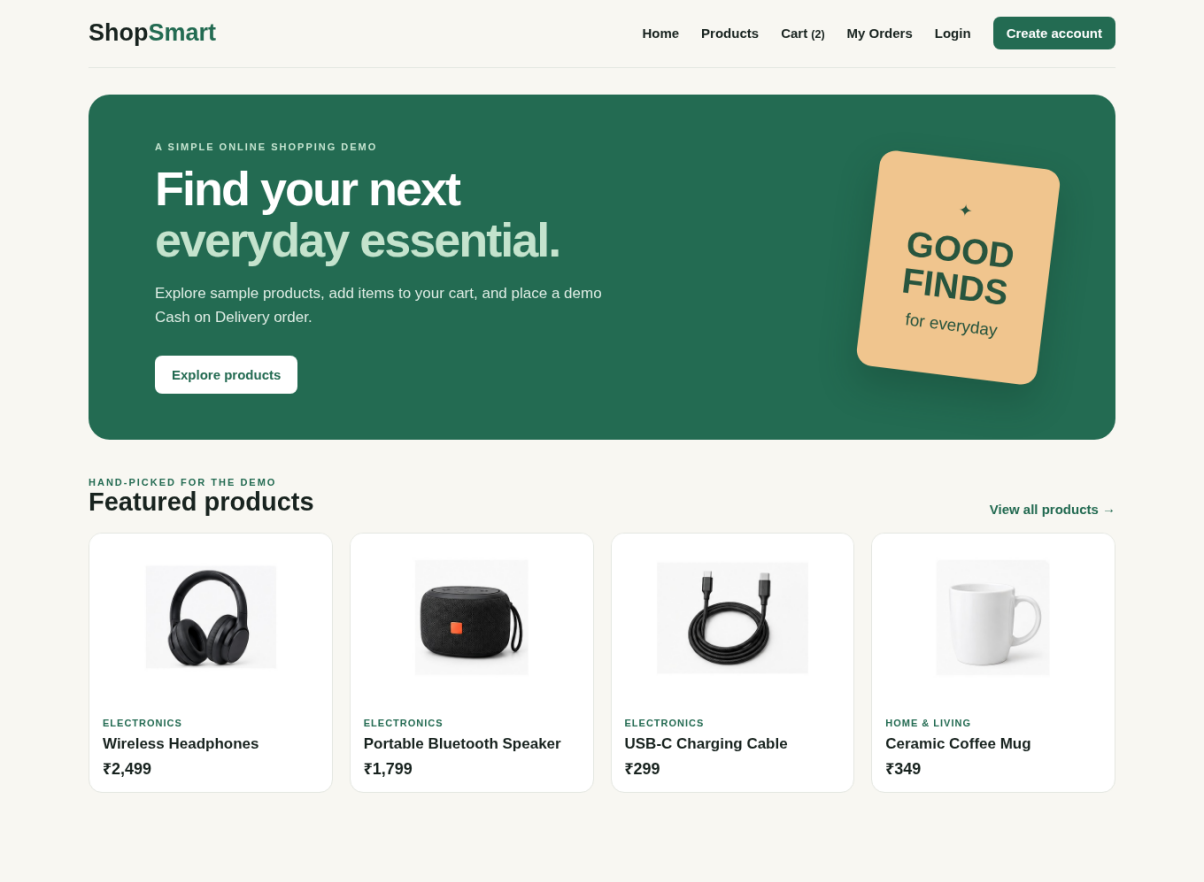


Figure 2. ShopSmart home page and featured product section.

The home page introduces the store and provides a starting point for product discovery. A featured-products section highlights sample items and directs visitors toward the wider catalogue.

---

13. ILLUSTRATIVE SCREENSHOT — PRODUCT CATALOGUE:-

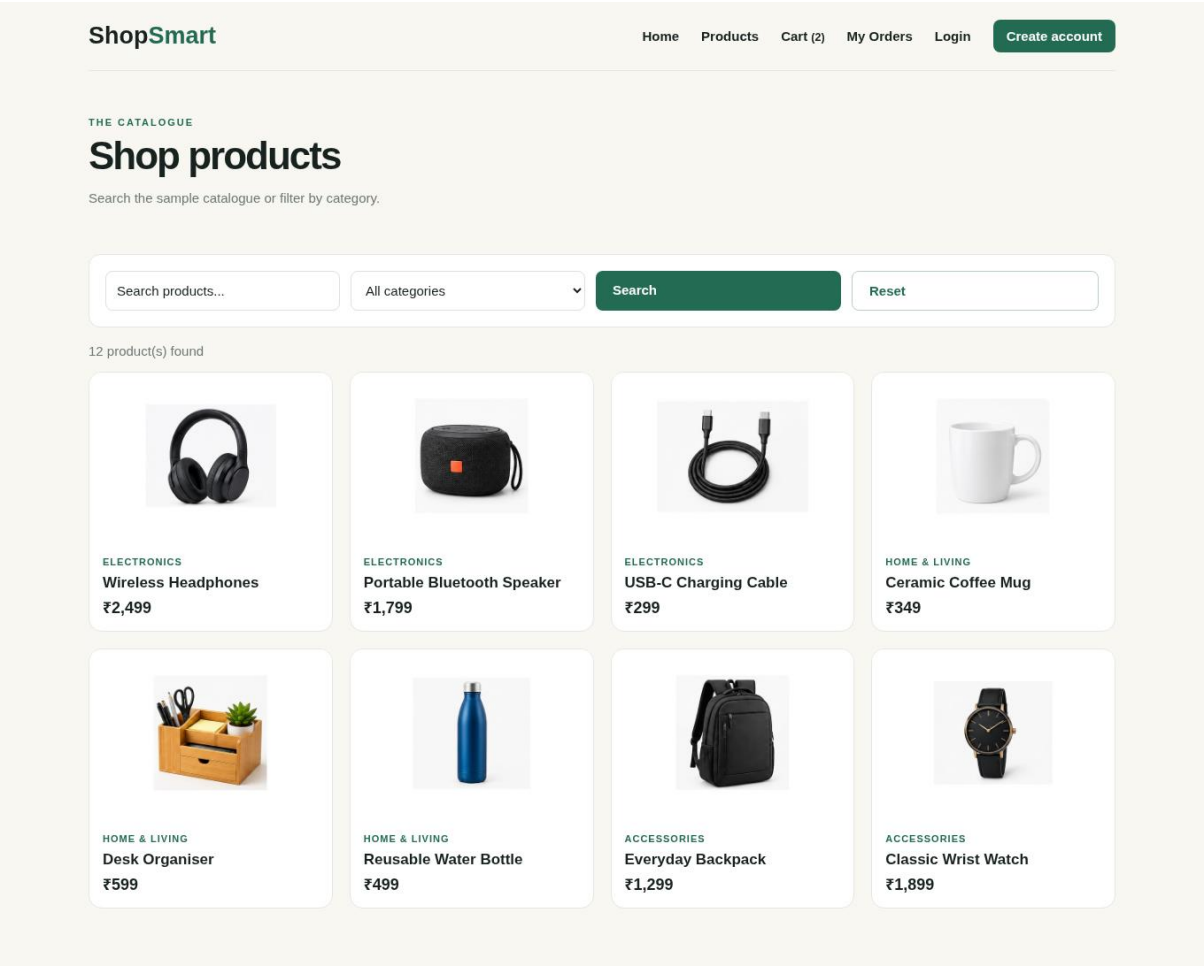


Figure 3. Product catalogue with search and category filtering.

The catalogue presents sample products in a grid. Search and category controls represents the expected product-discovery workflow. Product availability and active status are controlled by the database.

---

14. ILLUSTRATIVE SCREENSHOT — CUSTOMER LOGIN:-

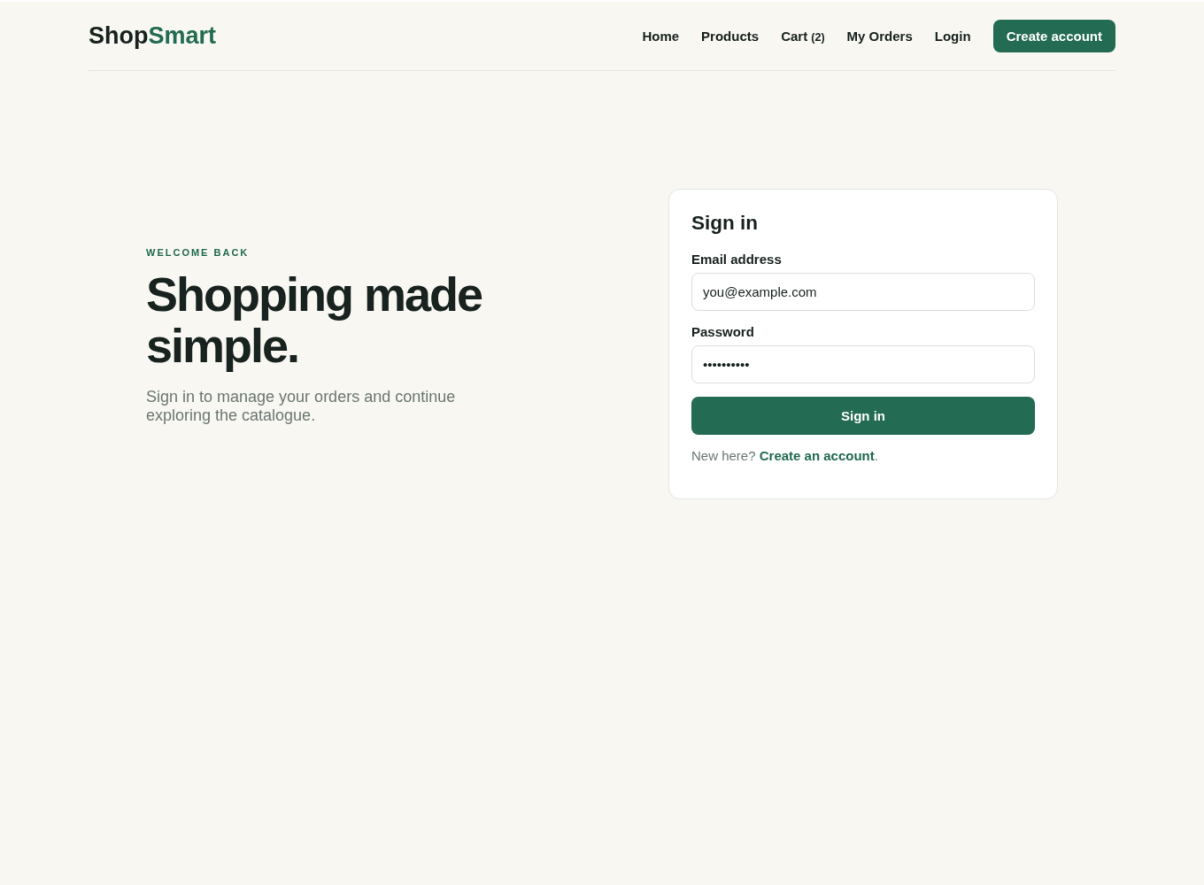


Figure 4. Customer sign-in interface.

The login screen collects an email address and password. The server verifies the password against the stored password hash. Successful login creates a session for the customer.

---

15. ILLUSTRATIVE SCREENSHOT — SHOPPING CART:-

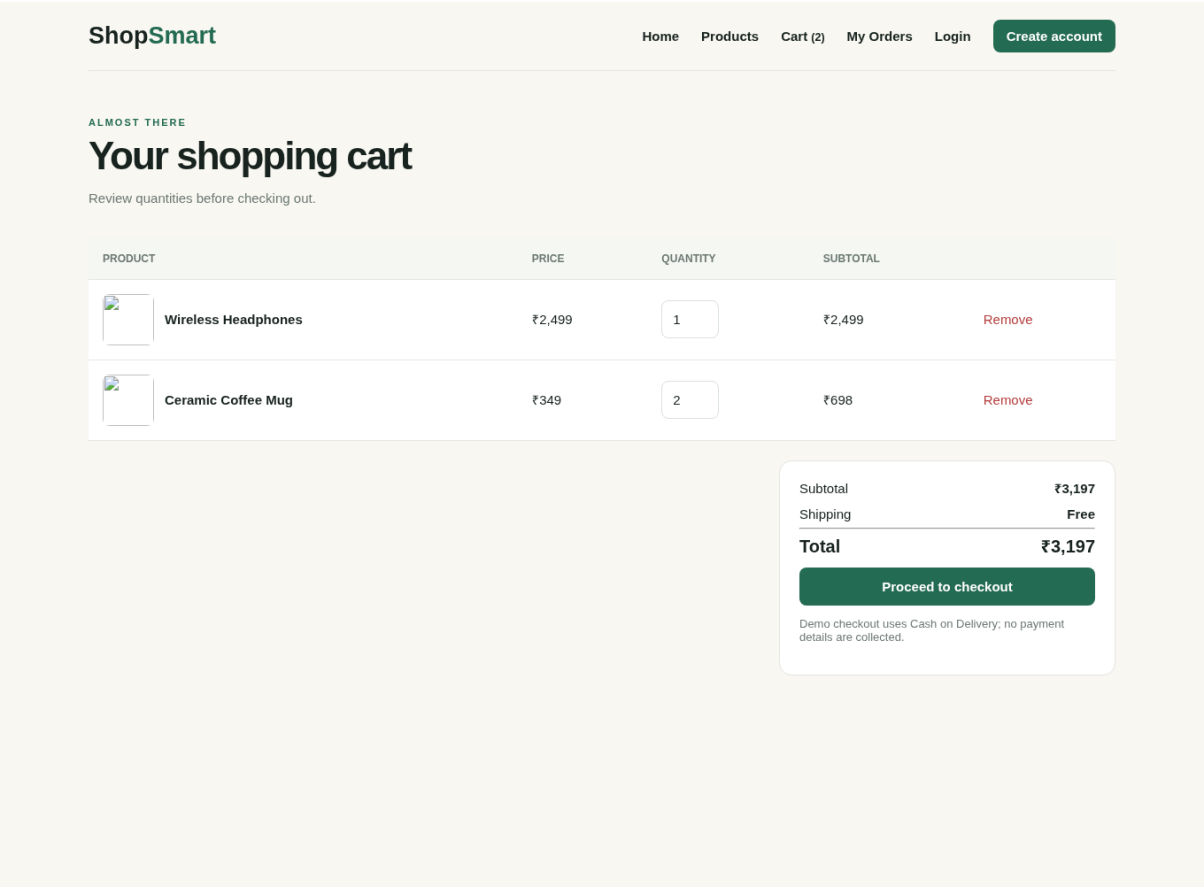


Figure 5. Shopping cart showing products, quantities and subtotal.

The cart allows the customer to review selected products, change quantities and remove unwanted items. The server recalculates totals using product data retrieved from the database.

---

16. ILLUSTRATIVE SCREENSHOT — COD CHECKOUT:-

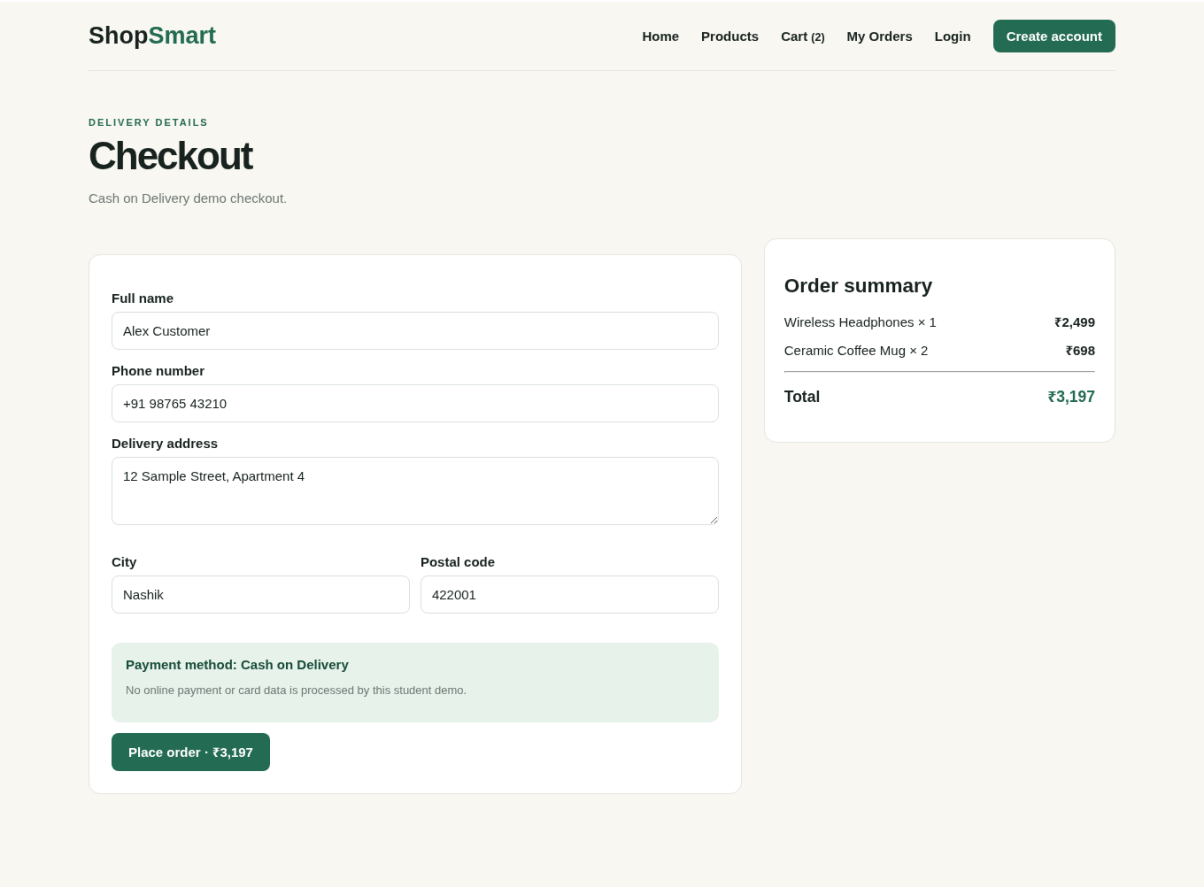


Figure 6. Delivery form and Cash on Delivery order summary.

Checkout collects the delivery contact and address details. The sample application supports Cash on Delivery only and does not request card numbers or online payment credentials.

---

17. ILLUSTRATIVE SCREENSHOT — ADMIN DASHBOARD:-

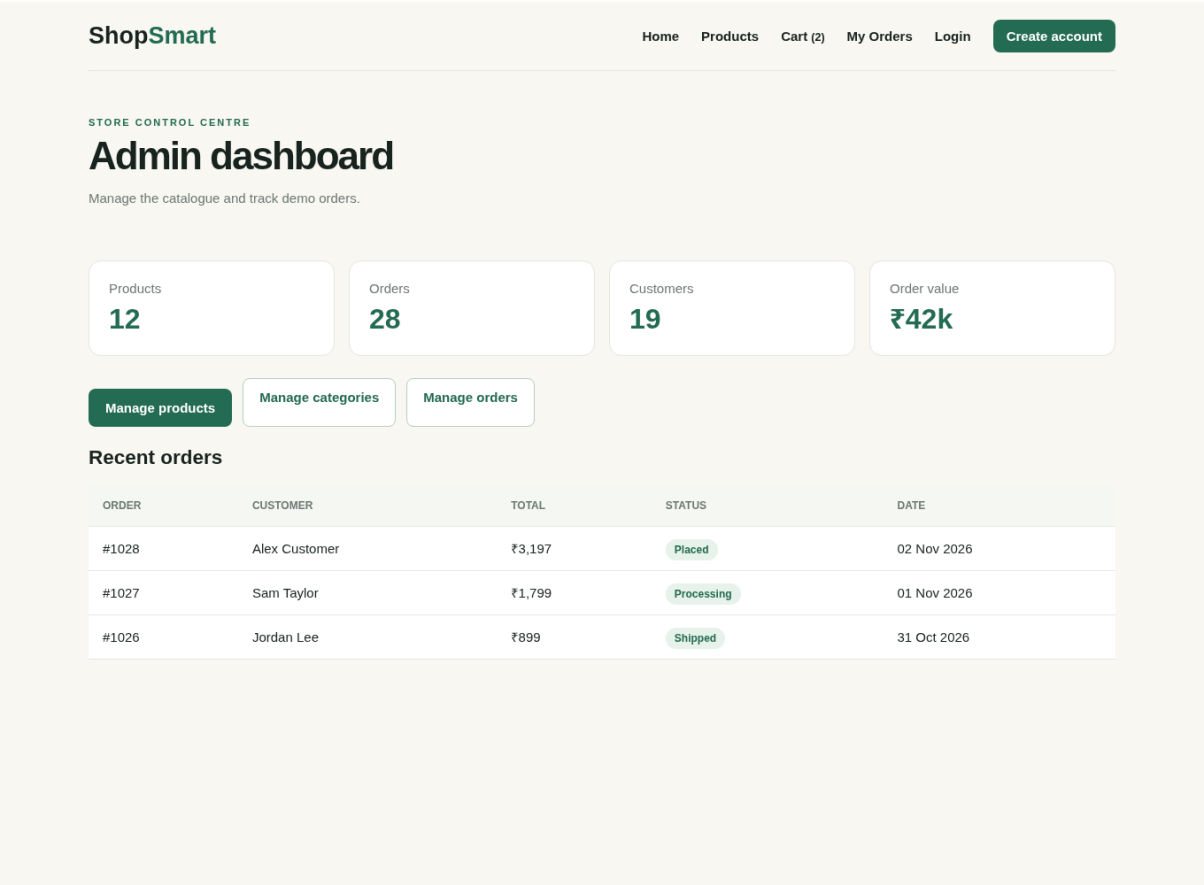


Figure 7. Admin dashboard with summary metrics and recent orders.

The dashboard summarizes products, order and customer counts and displays recent order information. The administrator can navigate to product, category and order management pages.

---

18. ILLUSTRATIVE SCREENSHOT — DATABASE DIAGRAM:-

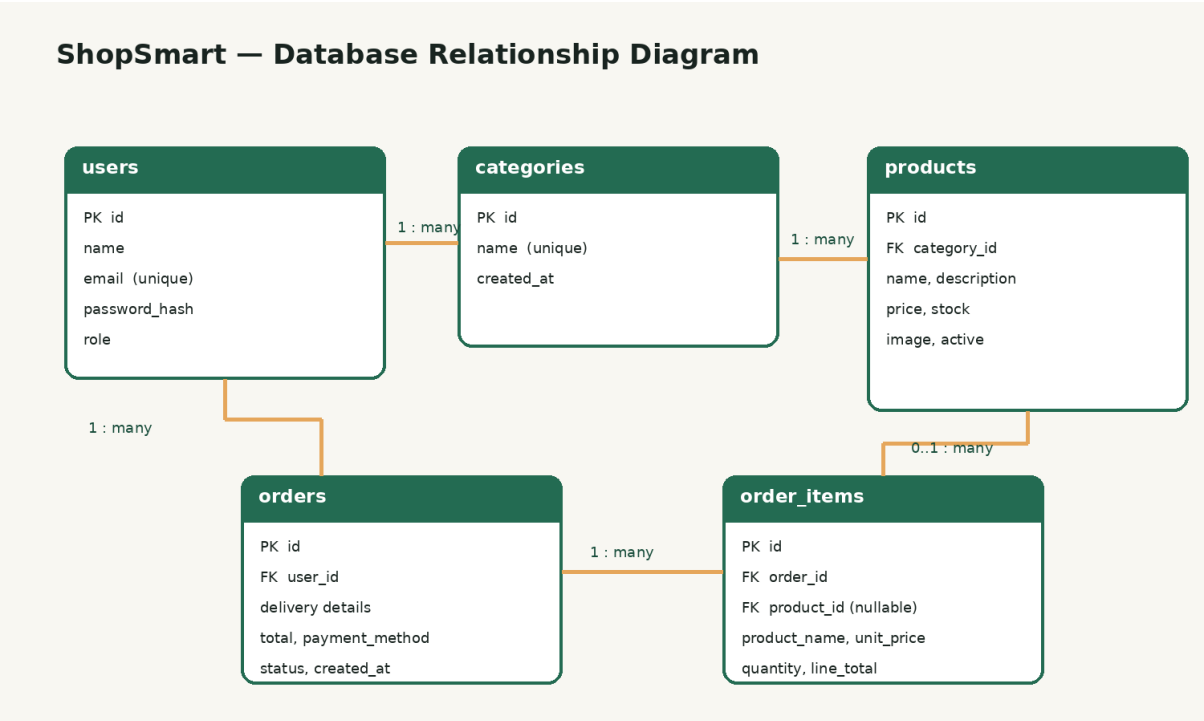


Figure 8. Database relationship view.

The database diagram shows the main entities and their one-to-many relationships. Refer to the SQL script for exact column types, indexes, foreign-key rules and allowed status values.

This image is an illustrative diagram prepared for documentation.

---

19. SECURITY, VALIDATION AND TESTING PLAN:-

19.1 Security Measures in Supplied Code

- PDO prepared statements are used for database operations involving input values.
- Password hashes are stored and checked using password verification functions.
- HTML output is escaped through a shared helper to reduce cross-site scripting risk.
- POST operations verify a CSRF token.
- Session ID regeneration is performed after successful login.
- Admin pages apply a role-based access guard.
- Checkout users a database transaction and locks products rows while checking stock.
- Server-side validation checks checkout name, phone, address, city and postal code.

19.2 Manual Testing Checklist

Test	Expected result	Evidence
Home page	Page and featured products load	Capture browser screenshot
Search/filter	Matching products appear	Record search and filter input
Registration/login	Valid account signs in; invalid password rejected	Record test outcome
Cart	Add, update and remove actions work	Capture cart status
Checkout	Valid COD order is created and stock reduced	Record order ID and stock
Admin guard	Customer denied; admin allowed	Capture both outcomes

Testing status: Not executed in the report-generation environment. Perform these tests locally and record actual Pass/Fail results before submission.

---

20. INSTALLATION, EXECUTION AND TROUBLESHOOTING:-

1. Install XAMPP and start Apache and MySQL.
2. Extract E-Commerce-Website.zip.
3. Copy the E-Commerce-Website folder into C:\xampp\htdocs\.
4. Open http://localhost/phpmyadmin and import database/ecommerce.sql.
5. Check config/database.php. Defaults: host 127.0.0.1, database ecommerce_portal, username root and blank password. Change these if your local setup differs.
6. Open http://localhost/E-Commerce-Website/ in Chrome or Edge. PHP pages must be opened through localhost, not file://.
7. Register a customer account and test sign-in, catalogue and cart.
8. To create an admin locally, run UPDATE users SET role='admin' WHERE email='your-email@example.com'; using the email of your own registered account. Log out and sign in again.

Problem	Suggested check
Database connection error	Ensure MySQL is running; confirm database import and credentials.
404 Not Found	Check folder location under htdocs, URL spelling and Apache status.
Admin link missing	Check role in users table, then sign out and sign in again.
Product image missing	Check images paths and confirm the assets/images files exist.
Port conflict	Close conflicting service or change the XAMPP port configuration.

---

21. LIMITATIONS FUTURE SCOPE, CONCLUSION AND REFERENCES:-

21.1 Current Limitations

- The checkout supports COD only; no real payment gateway is connected.
- The catalogue contains sample product records and sample images.
- The project is intended for local academic demonstration, not production deployment.
- Advanced analytics, customer reviews, shipping integration and automated email notifications are outside the current scope.

21.2 Future Enhancement

- Integrate a payment gateway with secure server-side verification.
- Add email verification, password reset and login rate limiting.
- Add wish lists, coupons, product reviews and advanced filters.
- Add scales charts, downloadable reports and low-stock alerts.
- Integrate shipping services and customer order notifications.
- Harden production deployment with HTTPS, secure cookies, backups, privacy notices and audit logs.

21.3 Conclusion

ShopSmart demonstrates the essential components of a database-backed e-commerce website: product browsing, customer accounts, a session-based and MySQL structure provides a useful foundation for learning web application development and relational database design. Local execution and testing are required before presenting the application as fully verified.

21.4 References

- Supplied source archive: E-Commerce-Website.zip, including README.md and PHP source files.
- Supplied SQL schema and data: database/ecommerce.sql.
- PHP documentation: https://www.php.net/docs.php
- MySQL documentation: https://dev.mysql.com/doc/
- XAMPP: https://www.apachefriends.org/
- Visual Studio Code: https://code.visualstudio.com/docs

Final submission reminder: replace illustrative interface designs with actual screenshots captured from your own running application if required by your college. Fill in student, guide, institution and academic-year details.